

Healthcare Data Warehouse & Analytics Platform

Developer Guide

A Beginner-Friendly, Step-by-Step Technical Documentation

Version 1.0



Digital Egypt Pioneers Initiative (DEPI)
AI & Data Science Track — Microsoft Data Engineer

Prepared by

Mahmoud Salem

<https://linkedin.com/in/ma7moudalysalem>

Version 1.0 — February 25, 2026

Contents

1	Introduction	6
1.1	What Is This Project?	6
1.2	Who Is This Guide For?	7
1.3	Technologies at a Glance	7
1.4	How to Read This Guide	9
2	Project Structure — Every Folder Explained	10
2.1	The Big Picture	10
2.2	Folder-by-Folder Breakdown	11
2.2.1	src/ — The Source Code	11
2.2.2	notebooks/ — Jupyter and Databricks Notebooks	12
2.2.3	sql/ — All SQL Scripts	12
2.2.4	pipelines/ — Data Pipeline Templates	13
2.2.5	dashboards/ — Visualization Files	13
2.2.6	docs/ — Documentation	13
2.2.7	tests/ — Automated Tests	13
2.2.8	docker/ — Container Configuration	14
2.2.9	data/ — Data Files (Git-Ignored)	14
2.3	Root-Level Files Explained	14
3	The Architecture — How Data Flows	16
3.1	The Medallion Architecture	16
3.1.1	Bronze Layer — Raw Data Landing	17
3.1.2	Silver Layer — Cleaned and Validated	17
3.1.3	Gold Layer — Business-Ready	18
3.2	The Complete Data Flow	18
3.2.1	Batch Path (Step by Step)	19
3.2.2	Streaming Path (Step by Step)	19
4	Data Modeling — The Database Design	22
4.1	Two Databases, Two Purposes	22
4.2	OLTP Source Schema (3NF)	23
4.2.1	Source Tables	23
4.2.2	Entity-Relationship Diagram	24
4.3	Star Schema (Data Warehouse)	25

4.3.1	Fact Tables	25
4.3.2	Dimension Tables	26
4.3.3	Surrogate Keys vs. Business Keys	28
4.3.4	Synapse Distribution Strategies	28
5	Data Sources — Where the Data Comes From	31
5.1	Source 1: Synthea (Patient Records)	31
5.2	Source 2: Custom Claims Generator (Python + Faker)	32
5.3	Source 3: Vital Signs Simulator (IoT Stream)	32
5.4	Source 4: Provider API (Flask REST API)	33
6	Docker — Running the Local Environment	35
6.1	What Is Docker and Why Do We Use It?	35
6.2	Our Docker Setup	36
6.2.1	docker-compose.yml — Defining Our Services	36
6.2.2	Dockerfile.generator — Building the Python Image	38
6.3	How to Use Docker in This Project	39
7	Configuration — Environment Variables and Dependencies	41
7.1	Why Environment Variables?	41
7.2	The .env.example File	42
7.3	Python Dependencies (requirements.txt)	43
7.4	Code Quality Configuration (setup.cfg)	44
8	CI/CD Pipeline — Automated Quality Checks	46
8.1	What Is CI/CD?	46
8.2	Our Pipeline Configuration	46
8.3	Pipeline Flow Explained	48
8.3.1	Job 1: Python Linting (flake8)	48
8.3.2	Job 2: Unit Tests (pytest)	48
8.3.3	Job 3: SQL Validation (sqlfluff)	48
9	Testing — Verifying the Project Works	50
9.1	Why Test?	50
9.2	Our Test Suite	51
9.2.1	Understanding Each Test	53
9.3	Running Tests Locally	53
10	Git and Version Control	55
10.1	Git Basics for This Project	55
10.2	The .gitignore File	56

10.3 The .gitattributes File	57
10.4 Common Git Workflow	57
11 Analytical SQL Queries	59
11.1 Key SQL Concepts Explained	60
11.2 Deep Dive: 30-Day Readmission Rate	62
12 Azure Cloud Services — What Each Service Does	65
12.1 Azure Data Lake Storage Gen2	65
12.2 Azure Data Factory (ADF)	65
12.3 Azure Databricks	66
12.4 Azure Event Hubs	67
12.5 Azure Synapse Analytics	67
12.6 How Services Connect	68
13 Getting Started — Step-by-Step Setup	69
13.1 Prerequisites	69
13.2 Step 1: Clone the Repository	70
13.3 Step 2: Create a Virtual Environment	70
13.4 Step 3: Install Python Dependencies	71
13.5 Step 4: Configure Environment Variables	71
13.6 Step 5: Start Docker Services	71
13.7 Step 6: Run Tests	71
13.8 Step 7: Run the Data Generators	72
13.9 Quick Reference Card	73
14 Project Milestones and Timeline	74
14.1 Milestone 1: Data Collection & EDA (Weeks 1–2)	74
14.2 Milestone 2: Data Modeling & Warehouse (Weeks 3–4)	75
14.3 Milestone 3: Pipelines — Batch + Stream (Weeks 5–7)	75
14.4 Milestone 4: BI Dashboards (Week 8)	75
14.5 Milestone 5: DevOps & Presentation (Weeks 9–10)	76
15 Team and Contribution Guidelines	77
15.1 Team Members	77
15.2 How to Contribute	77
16 Conclusion and Learning Roadmap	79
16.1 What You Have Learned	79
16.2 Your Learning Roadmap	80
16.2.1 Level 1: Foundations (Start Here)	80

16.2.2 Level 2: Core Data Engineering	80
16.2.3 Level 3: Cloud and Production	81
16.2.4 Level 4: Advanced Topics	81
16.3 Final Words	81
17 Glossary	83

List of Figures

3.1	The Medallion Architecture: Bronze → Silver → Gold	16
3.2	Complete data flow — batch (top) and streaming (bottom) paths	19
4.1	OLTP Entity-Relationship Diagram — arrows point from child to parent (FK direction)	24
8.1	CI Pipeline execution flow	48
12.1	Azure service interconnections	68
14.1	Project timeline — 5 milestones over 10 weeks	74

Chapter 1

Introduction

Learning Objectives

By the end of this chapter, you will be able to:

- Explain what a data warehouse is and how it differs from an operational database
- Describe the purpose and scope of this project
- Identify all the technologies used and what role each one plays
- Understand the difference between OLTP and OLAP systems

1.1 What Is This Project?

Imagine a hospital network with **5 hospitals and 20+ clinics** spread across Egypt. Every day, thousands of patients visit these facilities. Their data — medical records, lab results, insurance claims, vital signs — is scattered across many different computer systems that do not talk to each other.

This is a common real-world problem called **data silos**: each system stores its own data in its own format, making it nearly impossible to get a complete picture of what is happening across the entire network.

This project builds a **centralized data warehouse** that:

1. **Collects** all that scattered data into one place
2. **Cleans and organizes** it so it is consistent and reliable
3. **Transforms** it into a format optimized for analytics
4. **Presents** it through interactive dashboards for decision-makers
5. **Monitors** real-time vital signs and raises alerts for anomalies

What Is a Data Warehouse?

A **data warehouse** is a special database designed for *analyzing* data, not for running daily operations. Think of it this way:

- **OLTP (Operational Database)** = The cash register at a store. It handles individual transactions quickly: “Patient X visited Doctor Y on this date.”
- **OLAP (Data Warehouse)** = The accountant’s summary report. It answers big-picture questions: “How many patients were readmitted within 30 days across all hospitals this quarter?”

Here is a simple comparison to make this concrete:

Aspect	OLTP (Cash Register)	OLAP (Accountant)
Typical query	“Get patient #123’s record”	“Average cost per diagnosis across all hospitals”
Speed priority	Fast writes (milliseconds)	Fast reads over millions of rows
Data scope	Current data	Historical data (months/years)
Users	Doctors, nurses, apps	Analysts, managers, dashboards

1.2 Who Is This Guide For?

This guide is written for **students and junior developers** who are:

- Learning data engineering for the first time
- Familiar with basic programming (Python, SQL) but new to cloud and big data
- Joining this project or building something similar

We explain **every concept before showing you the code**, so you always understand *why* before *how*. No prior knowledge of cloud computing, data warehousing, or healthcare IT is assumed.

1.3 Technologies at a Glance

Before diving in, here is every technology we use and *what it does* in plain English:

Technology	Category	What It Does (Simply)
Python 3.11	Language	The main programming language for scripts and data processing
T-SQL	Language	Microsoft's flavor of SQL for querying databases
PySpark	Language	Python + Apache Spark for processing huge datasets
DAX	Language	Formula language for Power BI calculations
SQL Server 2022	Database	Stores raw patient data locally (like a local filing cabinet)
Azure Synapse	Database	Cloud data warehouse for analytics (the big filing room)
Azure Data Lake	Storage	Stores files (CSV, JSON, Parquet) in the cloud cheaply
Azure Data Factory	Orchestration	Moves data between systems on a schedule (like a delivery truck)
Azure Databricks	Processing	Runs Spark jobs to clean and transform data (the factory)
Azure Event Hubs	Streaming	Receives real-time data streams (like a mail slot)
Docker	DevOps	Packages software into containers for consistent environments
GitHub Actions	DevOps	Automatically tests code every time you push changes
Git	DevOps	Version control — tracks all changes to your code
Power BI	Visualization	Creates interactive business dashboards
Streamlit	Visualization	Python framework for building simple web dashboards
Synthea	Data Generation	Generates realistic (but fake) patient medical records
Faker	Data Generation	Python library for generating fake data (names, dates, etc.)

 Tip

Do not worry about memorizing all these technologies now. Each chapter introduces the relevant tools *when you need them*. This table is here as a reference you can come back to at any time.

1.4 How to Read This Guide

This guide is structured to be read **in order**, with each chapter building on the previous one. However, if you already know a topic, feel free to skip ahead. Here is the recommended path:

1. **Chapters 1–2:** Understand the project scope and folder structure
2. **Chapters 3–4:** Learn the architecture and data model (the “blueprint”)
3. **Chapter 5:** Understand where our data comes from
4. **Chapters 6–8:** Development environment (Docker, config, CI/CD)
5. **Chapters 9–10:** Testing and Git workflow
6. **Chapters 11–12:** SQL analytics and Azure cloud services
7. **Chapters 13–15:** Getting started, milestones, and team guidelines
8. **Chapter 16:** Conclusion and your learning roadmap
9. **Glossary:** Quick-reference for any unfamiliar terms

 Chapter Summary**Key Takeaways:**

- This project builds a centralized data warehouse for a hospital network using Azure cloud services
- A data warehouse (OLAP) is optimized for analytics, unlike operational databases (OLTP) which handle daily transactions
- The project uses Python, SQL, PySpark, Docker, Azure services, and Power BI
- This guide explains every concept from first principles — no prior cloud or data warehousing experience needed

Chapter 2

Project Structure — Every Folder Explained

Learning Objectives

By the end of this chapter, you will be able to:

- Navigate the project directory with confidence
- Explain the purpose of every folder and root-level file
- Understand Python package conventions (`__init__.py`)
- Know which files are tracked by Git and which are ignored

When you first open this project, you will see many folders and files. Do not worry — each one has a clear purpose. Let us go through **every single one**.

2.1 The Big Picture

```
healthcare-data-warehouse/
|-- src/                    # All Python source code
|   |-- generators/         # Scripts that create fake data
|   +-- synthea/            # Synthea config files
|-- notebooks/              # Jupyter and Databricks notebooks
|-- sql/                    # All SQL scripts
|   |-- ddl/                # CREATE TABLE statements
|   |-- queries/            # Analytical queries
|   +-- stored_procedures/  # Reusable SQL procedures
|-- pipelines/
|   +-- adf_templates/      # Azure Data Factory templates
|-- dashboards/             # Power BI files and screenshots
|   +-- screenshots/        # Dashboard images
|-- docs/                   # Documentation
|   |-- architecture/       # Architecture diagrams
```

```
|   +-- data_dictionary/      # Column-level documentation
|-- tests/                   # Automated test files
|-- docker/                  # Docker configuration
|-- data/                    # Data files (NOT in git)
|   |-- raw/                 # Raw CSV/JSON files
|   |-- cleaned/             # Cleaned Parquet files
|   +-- gold/                # Final analytics-ready files
|-- config/                  # App configuration files
|-- .github/workflows/       # CI/CD pipeline config
|-- requirements.txt          # Python dependencies
|-- .env.example              # Environment variable template
|-- setup.cfg                 # Linting and test config
|-- docker-compose.yml        # (inside docker/)
+-- README.md                 # Project overview
```

Listing 2.1: Complete project directory tree

Note

The tree above uses a common convention: `|-` means “this item is inside the folder above it,” and `+-` means “this is the last item in that folder.” Lines with `#` are comments explaining what each folder contains.

2.2 Folder-by-Folder Breakdown

2.2.1 `src/` — The Source Code

This is where all the Python code lives. It has two subfolders:

- `src/generators/` — Contains Python scripts that **generate fake healthcare data**:
 - `generate_claims.py` — Creates 200,000 fake insurance claims
 - `simulate_vitals.py` — Simulates IoT vital sign readings in real-time
 - `provider_api.py` — A Flask web API that serves doctor/provider data
- `src/synthea/` — Configuration for **Synthea**, a tool that generates realistic patient records

 Tip

Every Python package (folder) has an `__init__.py` file. This is a Python convention that tells Python: “This folder is a package, you can import from it.” Even if the file is empty, it must exist. Without it, Python cannot use `import` statements to load code from that folder.

2.2.2 notebooks/ — Jupyter and Databricks Notebooks

Notebooks combine code, output, and explanations in one document. We use them for:

- **Exploratory Data Analysis (EDA)** — Analyzing the generated data with charts and statistics
- **Databricks PySpark notebooks** — Running Spark transformations in the cloud

What Is a Jupyter Notebook?

A **Jupyter Notebook** is an interactive document that contains:

- **Code cells** — blocks of Python code you can run one at a time
- **Output cells** — the results (tables, charts, text) appear directly below the code
- **Markdown cells** — formatted text for explanations and documentation

Think of it as a lab notebook where you can write notes *and* run experiments in the same document. Files end with `.ipynb` (short for “Interactive Python Notebook”).

2.2.3 sql/ — All SQL Scripts

- **sql/ddl/** — **DDL** stands for “Data Definition Language.” These are `CREATE TABLE` statements that define the structure of our tables — columns, data types, constraints, indexes.
- **sql/queries/** — 10+ analytical queries that answer business questions (e.g., “What is the 30-day readmission rate?”)
- **sql/stored_procedures/** — Reusable SQL code blocks stored inside the database

DDL vs DML — What Is the Difference?

- **DDL (Data Definition Language):** Commands that *define structure* — `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`

- **DML (Data Manipulation Language):** Commands that *work with data* — SELECT, INSERT, UPDATE, DELETE

Think of DDL as building the shelves, and DML as putting books on those shelves.

Simple example:

```
-- DDL: Build the shelf
CREATE TABLE patients (
    patient_id INT PRIMARY KEY,
    name VARCHAR(100)
);

-- DML: Put books on the shelf
INSERT INTO patients VALUES (1, 'Ahmed Ali');
SELECT * FROM patients;
```

2.2.4 pipelines/ — Data Pipeline Templates

Contains Azure Data Factory (ADF) ARM templates — these are JSON files that define automated data movement workflows. Think of them as “recipes” that tell Azure: “Every day at 2 AM, copy new files from the Data Lake into Databricks for processing.”

2.2.5 dashboards/ — Visualization Files

- Power BI .pbix files (the actual dashboards)
- Streamlit Python app (optional web-based dashboard)
- Screenshots of the dashboards

2.2.6 docs/ — Documentation

- docs/architecture/ — Visual diagrams showing how data flows through the system
- docs/data_dictionary/ — Detailed description of every column in every table

2.2.7 tests/ — Automated Tests

Contains `pytest` test files that verify the project is set up correctly. We explain these in detail in Chapter 9.

2.2.8 docker/ — Container Configuration

Contains files that let you run the entire development environment in Docker containers. More on this in Chapter 6.

2.2.9 data/ — Data Files (Git-Ignored)

This folder holds actual data files, but they are **not tracked by Git** because:

- Data files can be **very large** (gigabytes)
- They can be **regenerated** by running the generator scripts
- They might contain **sensitive information** in real projects

The three subfolders correspond to the **Medallion Architecture** layers (explained in Chapter 3).

2.3 Root-Level Files Explained

File	Purpose
README.md	The “front page” of the project on GitHub. Written in Markdown.
requirements.txt	Lists all Python packages the project needs.
.env.example	A template showing what environment variables to set.
.gitignore	Tells Git which files to NOT track (data, secrets, caches).
.gitattributes	Ensures consistent line endings across Windows/Mac/Linux.
setup.cfg	Configuration for code linting (flake8) and testing (pytest).

Try It Yourself

Practice: Open the project in your file explorer or terminal and:

1. List all files in the `src/generators/` folder. How many Python files are there?
2. Open `.gitignore` and find which file patterns are ignored. Why is `.env` ignored but `.env.example` is not?
3. Look at the `data/` folder. Is it empty? Why does it exist in Git even though

its contents are ignored? (*Hint: look for `.gitkeep` files*)

✓ Chapter Summary

Key Takeaways:

- The project follows a clean, modular folder structure separating code (`src/`), data (`data/`), tests (`tests/`), docs (`docs/`), and config files
- Each Python package folder contains an `__init__.py` file
- Data files and secrets (`.env`) are excluded from Git using `.gitignore`
- The `data/` subfolders (raw, cleaned, gold) mirror the Medallion Architecture layers

Chapter 3

The Architecture — How Data Flows

Learning Objectives

By the end of this chapter, you will be able to:

- Explain the Medallion Architecture and why it uses three layers
- Describe what happens at each layer (Bronze, Silver, Gold)
- Trace the complete data flow from source to dashboard (both batch and streaming)
- Understand the difference between batch and streaming data processing
- Define key terms: Delta Lake, ACID transactions, schema enforcement

3.1 The Medallion Architecture

Our project uses the **Medallion Architecture**, a popular pattern in modern data engineering. It organizes data into three layers, each progressively cleaner and more useful:

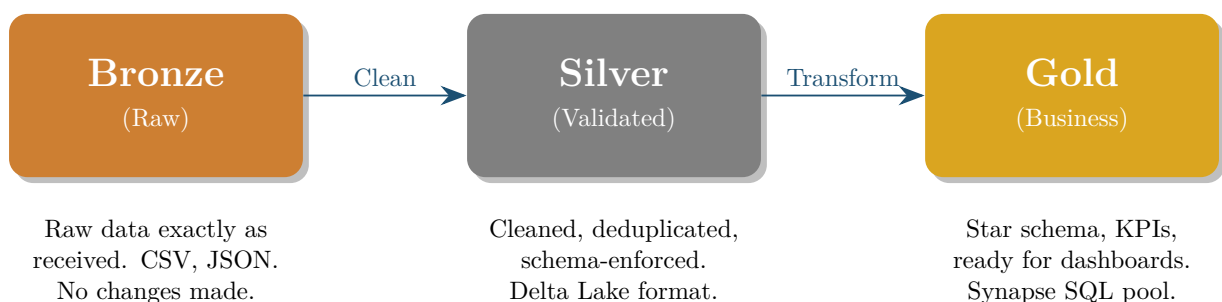


Figure 3.1: The Medallion Architecture: Bronze → Silver → Gold

Why Three Layers?

Think of it like cooking:

1. **Bronze** = Raw ingredients from the market (unwashed vegetables, uncut meat). You store them as-is so you always have the original.

2. **Silver** = Washed, peeled, and sliced ingredients. Clean and ready to combine, but not yet a dish.
3. **Gold** = The finished meal, plated and ready to serve. This is what the business consumers (dashboards, analysts) eat.

Why not just go straight from raw to final? Because if you make a mistake in cooking, you can go back to the cleaned ingredients (Silver) instead of starting from the raw market produce (Bronze) all over again. Each layer acts as a **checkpoint** you can return to.

3.1.1 Bronze Layer — Raw Data Landing

- **What goes here:** Raw CSV files from Synthea, JSON from the claims generator, streaming data from IoT sensors
- **Where it lives:** Azure Data Lake Gen2, container named **bronze**
- **Rule:** *Never modify data in Bronze.* It is your backup, your “single source of truth”
- **Data formats:** CSV, JSON, raw Parquet

Warning

The Bronze layer is **immutable** (read-only after writing). This is critical because if a bug in your cleaning code corrupts the Silver layer, you can always re-process from Bronze. Treat Bronze like a backup tape — write once, read many times, never edit.

3.1.2 Silver Layer — Cleaned and Validated

- **What happens here:**
 - Remove duplicate records
 - Handle missing values (nulls)
 - Enforce correct data types (dates are dates, numbers are numbers)
 - Normalize formats (all dates become ISO 8601: YYYY-MM-DD)
 - Standardize drug names using RxNorm codes
- **Where it lives:** Databricks Delta Lake tables, container named **silver**
- **Data format:** Delta Lake (Parquet + transaction log)

What Is Delta Lake?

Delta Lake is a storage format built on top of Parquet files. It adds:

- **ACID Transactions** — your writes either fully succeed or fully fail, never half-done
- **Schema Enforcement** — rejects data that does not match the expected columns/types
- **Time Travel** — you can query data as it looked yesterday or last week
- **Versioning** — every change creates a new version you can roll back to

Think of it as “Excel with an undo history that never loses anything.”

Simple example of Time Travel:

```
-- Query the current version of the table
SELECT * FROM silver.patients;

-- Query the table as it was 3 versions ago
SELECT * FROM silver.patients VERSION AS OF 3;

-- Query the table as it was yesterday
SELECT * FROM silver.patients TIMESTAMP AS OF '2024-01-15';
```

3.1.3 Gold Layer — Business-Ready

- **What happens here:** Data is reorganized into a **Star Schema** (explained in Chapter 4) with pre-calculated metrics
- **Where it lives:** Azure Synapse Analytics dedicated SQL pool
- **Who uses it:** Power BI dashboards, analysts writing SQL queries, Streamlit apps

3.2 The Complete Data Flow

Now that you understand the three layers, let us see how data actually moves through the system. There are two paths: **batch** (scheduled, large volumes) and **streaming** (real-time, continuous).

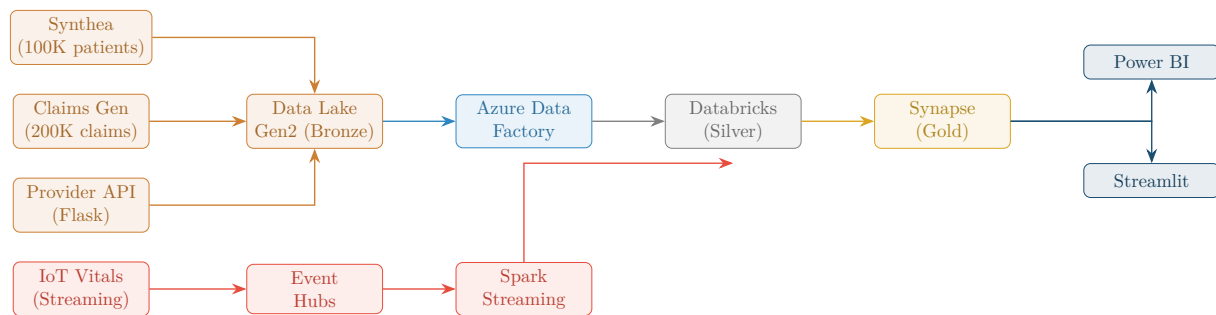


Figure 3.2: Complete data flow — batch (top) and streaming (bottom) paths

3.2.1 Batch Path (Step by Step)

Batch processing means processing data in large chunks on a schedule (e.g., every night at 2 AM), rather than one record at a time.

Step 1: Data Generation — Python scripts and Synthea generate fake healthcare data (CSV, JSON files)

Step 2: Land in Bronze — Files are uploaded to Azure Data Lake Gen2 in the **bronze** container, unchanged

Step 3: ADF Orchestration — Azure Data Factory triggers on a schedule, copying files from Bronze into Databricks

Step 4: Silver Processing — Databricks PySpark notebooks clean the data: deduplicate, validate schemas, normalize dates and drug codes, then save as Delta Lake tables

Step 5: Gold Transformation — More Databricks notebooks transform Silver data into a Star Schema: creating fact and dimension tables with surrogate keys

Step 6: Load to Synapse — Gold tables are loaded into Azure Synapse Analytics dedicated SQL pool

Step 7: Serve to Dashboards — Power BI connects to Synapse and renders interactive dashboards

3.2.2 Streaming Path (Step by Step)

Stream processing means processing data continuously as it arrives, with minimal delay (seconds to minutes).

Step 1: Vital Signs Simulation — A Python script simulates IoT devices sending heart rate, blood pressure, temperature, and SpO2 readings

- Step 2: Ingest via Event Hubs** — Readings flow into Azure Event Hubs (a high-throughput message queue)
- Step 3: Spark Structured Streaming** — Databricks reads from Event Hubs in real-time using Spark Structured Streaming
- Step 4: Anomaly Detection** — The stream processor checks for dangerous vitals (tachycardia, hypotension, fever) and flags them
- Step 5: Write to Delta Lake** — Processed readings are appended to a Delta Lake table (with an `anomaly_flag` column)
- Step 6: Real-Time Dashboards** — Power BI (DirectQuery mode) or Streamlit shows live vital signs and alerts

Batch vs. Streaming — When to Use Which?

Aspect	Batch	Streaming
Timing	Scheduled (hourly, daily)	Continuous (real-time)
Data volume	Large (millions of rows)	Small (one record at a time)
Latency	Minutes to hours	Seconds
Use case	Historical analysis, reports	Alerts, live monitoring
Our example	Patient records, claims	Vital sign readings

Most real-world systems use **both** — batch for heavy analytical workloads and streaming for time-sensitive alerts. This is called a **Lambda Architecture**.

Try It Yourself

Practice:

1. A patient visits a hospital and gets blood drawn. The lab results come back 2 hours later. Which path would these results follow — batch or streaming? Why?
2. A nurse attaches a heart rate monitor to a patient in the ICU. The monitor sends readings every 5 seconds. Which path? Why?
3. If you accidentally introduced a bug in the Silver cleaning code and it corrupted some dates, how would you fix it using the Medallion Architecture?

✓ Chapter Summary

Key Takeaways:

- The Medallion Architecture organizes data into Bronze (raw), Silver (cleaned), and Gold (business-ready) layers
- Bronze is immutable — never modify raw data so you can always reprocess from the source
- Delta Lake adds ACID transactions, schema enforcement, and time travel to Parquet files
- Batch processing handles large volumes on a schedule; streaming handles real-time data continuously
- Data flows: Sources → Data Lake (Bronze) → Databricks (Silver) → Synapse (Gold) → Dashboards

Chapter 4

Data Modeling — The Database Design

Learning Objectives

By the end of this chapter, you will be able to:

- Explain the difference between normalized (3NF) and denormalized (Star Schema) designs
- Read and interpret an Entity-Relationship (ER) diagram
- Identify fact tables and dimension tables in a Star Schema
- Define the grain of a fact table
- Explain Slowly Changing Dimensions (SCD) Types 1 and 2 with examples
- Understand Azure Synapse distribution strategies (HASH vs REPLICATE)

4.1 Two Databases, Two Purposes

This project uses **two different database designs** because they serve different needs:

	OLTP (Source)	OLAP (Warehouse)
Purpose	Handle daily operations	Analyze historical data
Design	3NF (Normalized)	Star Schema (Denormalized)
Optimized for	Writing (INSERT/UPDATE)	Reading (SELECT with JOINS)
Data volume	Current data	Years of historical data
Users	Applications, doctors	Analysts, dashboards
Our tool	SQL Server 2022	Azure Synapse Analytics

4.2 OLTP Source Schema (3NF)

The source system uses **Third Normal Form (3NF)**, which means data is organized to **minimize redundancy**. Each piece of information is stored in exactly one place.

What Is Normalization (3NF)?

Imagine you have a spreadsheet where every row of “encounters” repeats the patient’s name, address, and phone number. If the patient moves, you have to update hundreds of rows.

In **3NF**, you split this into separate tables:

- A `patients` table (name, address — stored once)
- An `encounters` table (references `patient_id` instead of repeating the name)

This is efficient for updates but requires **JOINS** when reading.

The Three Normal Forms (simplified):

1NF: Each column holds a single value (no lists or repeating groups)

2NF: Every non-key column depends on the *entire* primary key (not just part of it)

3NF: Every non-key column depends *only* on the primary key (not on other non-key columns)

4.2.1 Source Tables

Our OLTP database has 7 tables. Here is what each one stores:

Table	Primary Key	What It Stores
<code>patients</code>	<code>patient_id</code> (UUID)	Patient demographics: name, birth date, gender, race, city, state, insurance type
<code>providers</code>	<code>provider_id</code> (UUID)	Doctor/provider info: name, specialty, organization, city
<code>encounters</code>	<code>encounter_id</code> (UUID)	Hospital visits: type (inpatient/outpatient), start/end date, total cost. Links to patient and provider
<code>conditions</code>	<code>condition_id</code> (UUID)	Diagnoses: ICD-10 code, description, onset and resolved dates. Links to patient and encounter

Table	Primary Key	What It Stores
<code>medications</code>	<code>medication_id</code> (UUID)	Prescriptions: drug code (RxNorm), description, start/stop dates, cost. Links to patient and encounter
<code>claims</code>	<code>claim_id</code> (UUID)	Insurance claims: payer name, amount billed/paid, status (submitted/approved/denied/paid)
<code>vital_signs</code>	<code>vital_id</code> (BIGINT)	Real-time readings: heart rate, blood pressure (systolic/diastolic), temperature, SpO2, timestamp

💡 Tip

UUID stands for Universally Unique Identifier — a 128-bit value like 550e8400-e29b-41d4-a716-446655440000. It guarantees uniqueness across all systems without a central counter. UUIDs are ideal when multiple systems generate IDs independently (no two will ever collide).

4.2.2 Entity-Relationship Diagram

The following diagram shows how the OLTP tables relate to each other. Lines between tables represent **foreign key** relationships — a column in one table that references the primary key of another table.

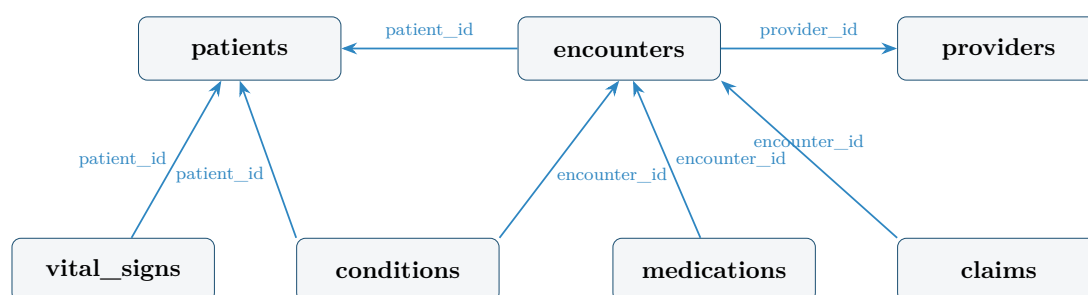


Figure 4.1: OLTP Entity-Relationship Diagram — arrows point from child to parent (FK direction)

📌 Note

Reading the diagram: An arrow from `encounters` to `patients` means “each encounter belongs to one patient” (the `encounters` table has a `patient_id` column that references `patients`). One patient can have many encounters — this is a **one-to-many** relationship.

4.3 Star Schema (Data Warehouse)

The warehouse uses a **Star Schema** — the most common design for analytics databases.

What Is a Star Schema?

A Star Schema has two types of tables:

- **Fact Tables** (center of the star) — contain *measurements/metrics*: costs, counts, amounts. Each row is an “event” or “transaction.”
- **Dimension Tables** (points of the star) — contain *descriptive attributes*: who, what, when, where. They give context to the facts.

It is called a “star” because when you draw the diagram, the fact table in the center connects to dimensions around it, forming a star shape.

Why not just use 3NF for analytics?

- 3NF requires many JOINS, which slows down large analytical queries
- Star Schema needs fewer JOINS (usually just fact → dimension)
- Business users find star schemas intuitive: “Give me *total cost* (fact) by *hospital* (dimension) for *Q1 2024* (dimension)”

A simple analogy:

- **Fact table** = A sales receipt (date, amount, product ID, customer ID)
- **Dimension tables** = Reference cards (product catalog, customer directory, calendar)
- To analyze sales, you join the receipt to the reference cards to get the full picture

4.3.1 Fact Tables

Table	Grain	Measures	Dimension Keys
<code>fact_encounters</code>	One row per hospital visit	<code>total_cost</code> , <code>length_of_stay</code> , <code>num_conditions</code> , <code>readmission_flag</code>	<code>patient_key</code> , <code>provider_key</code> , <code>date_key</code> , <code>payer_key</code>
<code>fact_claims</code>	One row per insurance claim	<code>amount_billed</code> , <code>amount_paid</code> , <code>amount_denied</code> , <code>processing_days</code>	<code>patient_key</code> , <code>counter_key</code> , <code>payer_key</code> , <code>date_key</code>
<code>fact_vitals</code>	One row per vital sign reading	<code>heart_rate</code> , <code>sys-tolic_bp</code> , <code>dias-tolic_bp</code> , <code>tem-perature</code> , <code>spo2</code> , <code>anomaly_flag</code>	<code>patient_key</code> , <code>date_key</code> , <code>time_key</code>

Note

The **grain** is the most important decision in fact table design. It answers: “What does one row represent?” Always define the grain *before* choosing columns. A wrong grain leads to incorrect aggregations and misleading reports.

Example: If the grain of `fact_encounters` is “one row per hospital visit,” then a patient who visited 3 times has 3 rows. The `total_cost` column holds the cost of *that specific visit*, not the patient’s total lifetime cost.

4.3.2 Dimension Tables

Table	SCD Type	Key Columns
<code>dim_patient</code>	Type 2	<code>patient_key</code> , <code>patient_id</code> , <code>name</code> , <code>birth_date</code> , <code>gender</code> , <code>race</code> , <code>city</code> , <code>state</code> , <code>insurance_type</code> , <code>valid_from</code> , <code>valid_to</code> , <code>is_current</code>
<code>dim_provider</code>	Type 2	<code>provider_key</code> , <code>provider_id</code> , <code>name</code> , <code>specialty</code> , <code>organization</code> , <code>valid_from</code> , <code>valid_to</code> , <code>is_current</code>

Table	SCD Type	Key Columns
dim_date	Static	date_key, full_date, year, quarter, month, week, day_of_week, is_weekend, is_holiday, fiscal_quarter
dim_time	Static	time_key, hour, minute, am_pm, shift (morning / afternoon / night)
dim_diagnosis	Type 1	diagnosis_key, icd10_code, description, category, body_system
dim_payer	Type 1	payer_key, payer_name, payer_type (private/public/self-pay), plan_name

What Are Slowly Changing Dimensions (SCD)?

Real-world data changes over time. A patient might move to a new city or change their insurance. How do you handle that?

SCD Type 1 — Overwrite:

- Simply update the old value with the new one
- You *lose* the history: you cannot see what the old value was
- Good for: corrections, typo fixes, attributes you do not need to track historically

Example: A diagnosis description has a typo. You just fix it:

diagnosis_key	description (before)	description (after)
501	Diabtes Type 2	Diabetes Type 2

SCD Type 2 — Add a New Row:

- Keep the old row (mark it as `is_current = 0`, set `valid_to = today`)
- Insert a new row with updated values (`is_current = 1`, `valid_from = today`)
- You *preserve* the full history
- Good for: patient address, insurance plan, provider specialty — anything you want to analyze historically

Example: Patient moves from Cairo to Alexandria on March 1:

patient_key	city	valid_from	valid_to	is_current
101	Cairo	2020-01-01	2024-02-28	0
102	Alexandria	2024-03-01	9999-12-31	1

Both rows have the same `patient_id` (the business key), but different `patient_key` (the surrogate key). An encounter on January 15 links to key 101 (Cairo), while an encounter on April 10 links to key 102 (Alexandria).

Why does this matter? If you analyze “average cost by city,” January encounters count toward Cairo and April encounters count toward Alexandria — giving you *historically accurate* results.

4.3.3 Surrogate Keys vs. Business Keys

What Is a Surrogate Key?

- A **business key** is the natural identifier from the source system (e.g., `patient_id` = UUID from the hospital)
- A **surrogate key** is an artificial integer generated by the warehouse (e.g., `patient_key` = 1, 2, 3, ...)

Why use surrogate keys instead of business keys?

1. **Performance:** Integer JOINS are faster than UUID/string JOINS
2. **SCD Type 2:** One `patient_id` can have multiple `patient_key` values (one per version)
3. **Source independence:** If the source system changes its ID format, the warehouse is unaffected

4.3.4 Synapse Distribution Strategies

Azure Synapse distributes data across 60 compute nodes. How you distribute matters for performance:

Strategy	How It Works	We Use It For
HASH	Rows with the same key value go to the same node	Fact tables (HASH on patient_key) — keeps related data together
REPLICATE	Full copy on every node	Dimension tables (they are small) — avoids shuffling during JOINS

💡 Tip

Rule of thumb: HASH-distribute large fact tables on the column you JOIN on most often (usually the main dimension key). REPLICATE small dimension tables so every node has a local copy and never needs to request data from other nodes.

✍ Try It Yourself

Practice:

1. A patient named Sara had insurance with “National Insurance Co.” until June 2024, then switched to “HealthPlus.” Draw the SCD Type 2 rows that would exist in `dim_patient` for Sara.
2. The `fact_encounters` table has the grain “one row per hospital visit.” If you wanted a table tracking “one row per medication prescribed,” what would you name it and what measures might it have?
3. Why is `dim_date` marked as “Static” (not SCD Type 1 or 2)? Think about whether dates ever change.

✅ Chapter Summary

Key Takeaways:

- OLTP uses 3NF (normalized) for efficient writes; OLAP uses Star Schema (denormalized) for efficient reads
- Star Schema consists of fact tables (measurements) surrounded by dimension tables (context)
- The grain defines what one row in a fact table represents — always define it first
- SCD Type 1 overwrites old values; SCD Type 2 preserves history with versioned rows

- Surrogate keys (integers) are used instead of business keys for performance and SCD support
- In Synapse: HASH-distribute large fact tables, REPLICATE small dimension tables

Chapter 5

Data Sources — Where the Data Comes From

Learning Objectives

By the end of this chapter, you will be able to:

- Explain why we use synthetic data instead of real patient data
- Describe each of the four data sources and what they produce
- Interpret medical coding standards (ICD-10, SNOMED CT, RxNorm)
- Understand normal vital sign ranges and what anomalies look like
- Explain what a REST API is and how the Provider API works

Since this is a capstone project, we cannot use real patient data (privacy laws like HIPAA forbid it). Instead, we generate **realistic synthetic data** from four sources.

5.1 Source 1: Synthea (Patient Records)

Synthea is an open-source tool by MITRE that generates realistic (but completely fictional) patient medical records.

- **Output:** CSV files for patients, encounters, conditions, medications, and more
- **Volume:** 100,000 patients, which generates ~500K encounters and 300K conditions
- **Medical accuracy:** Uses real clinical pathways — if a patient has diabetes, Synthea generates realistic medication sequences and complications
- **Standards:** Uses ICD-10 codes (diagnoses), SNOMED CT (clinical terms), RxNorm (medications)

Medical Coding Standards — Quick Reference

Hospitals worldwide use standardized codes so that different systems can understand each other:

- **ICD-10:** International Classification of Diseases. Example: E11.9 = Type 2 diabetes without complications. Used for billing and diagnosis tracking.
- **SNOMED CT:** A comprehensive clinical terminology. More detailed than ICD-10. Used in electronic health records.
- **RxNorm:** Standard names for medications. Example: “Metformin 500mg oral tablet” instead of brand names like “Glucophage.”

These standards ensure data is consistent and interoperable across hospital systems.

5.2 Source 2: Custom Claims Generator (Python + Faker)

The `generate_claims.py` script creates fake insurance claims using the **Faker** library.

What it generates:

- 200,000 insurance claims
- Each claim has: payer name, amount billed, amount paid, claim status
- Statuses: `submitted`, `approved`, `denied`, `appealed`, `paid`
- Realistic denial rate: 15–20% of claims are denied (matching real-world patterns)
- Multiple payer types: private insurance, public (government), self-pay

💡 Tip

Why realistic distributions matter: If 100% of claims were “approved,” our dashboards would show no useful insights. By modeling real-world patterns (15–20% denial rate, varying costs by encounter type), the data tells a realistic story that exercises all our analytical queries.

5.3 Source 3: Vital Signs Simulator (IoT Stream)

The `simulate_vitals.py` script simulates IoT medical devices sending patient vital signs in real-time.

Normal vital sign ranges:

Vital Sign	Normal Range	Unit	What It Measures
Heart Rate	60–100	bpm	How fast the heart beats
Systolic BP	90–120	mmHg	Pressure when heart contracts
Diastolic BP	60–80	mmHg	Pressure when heart relaxes
Temperature	36.5–37.5	°C	Core body temperature
SpO2	95–100	%	Oxygen level in the blood

Injected anomalies (for testing anomaly detection):

- **Tachycardia spikes** — Heart rate jumps above 120 bpm (fast heartbeat)
- **Hypotension episodes** — Systolic blood pressure drops below 90 mmHg (dangerously low)
- **Fever patterns** — Temperature exceeds 38.5°C

Note

The simulator deliberately injects about 5–10% anomalous readings so that the anomaly detection pipeline in Databricks has something to find. Without these injected anomalies, we would have nothing to test our alerting system against.

5.4 Source 4: Provider API (Flask REST API)

The `provider_api.py` runs a simple web API that serves doctor/provider information.

Endpoints:

Method	Endpoint	Returns
GET	<code>/providers</code>	List of all providers (JSON array)
GET	<code>/providers/{id}</code>	Details for a specific provider (JSON object)
GET	<code>/providers?specialty=cardiology</code>	Providers filtered by specialty

What Is a REST API?

A **REST API** (Representational State Transfer Application Programming Interface) is a way for programs to communicate over HTTP (the same protocol your web browser uses).

How it works:

1. Your program sends an HTTP **request** to a URL (e.g., `GET /providers`)

2. The server processes the request
3. The server sends back a **response** containing structured data (usually JSON)

Example: When you visit `http://localhost:5000/providers/123`, instead of getting an HTML web page, you get structured JSON data:

```
{
  "provider_id": "123",
  "name": "Dr. Ahmed Mohamed",
  "specialty": "Cardiology",
  "organization": "Cairo General Hospital"
}
```

This JSON data can be easily read by other programs, dashboards, or data pipelines.

✓ Chapter Summary

Key Takeaways:

- We use synthetic data because real patient data is protected by privacy laws (HIPAA)
- Synthea generates 100K realistic patients with proper medical coding (ICD-10, RxNorm)
- The claims generator creates 200K claims with realistic approval/denial patterns
- The vitals simulator produces real-time data with intentional anomalies for testing
- The Provider API serves data via REST endpoints that other systems can consume

Chapter 6

Docker — Running the Local Environment

📖 Learning Objectives

By the end of this chapter, you will be able to:

- Explain what Docker is and why it solves the “works on my machine” problem
- Define key Docker vocabulary: image, container, Dockerfile, volume
- Read and understand a `docker-compose.yml` file
- Read and understand a `Dockerfile`
- Start, stop, and debug the project’s Docker services

6.1 What Is Docker and Why Do We Use It?

Docker in Simple Terms

Imagine you wrote a Python script that works perfectly on your laptop. You give it to your teammate, and it crashes because they have a different Python version, missing libraries, or a different operating system.

Docker solves this by packaging your application *with everything it needs* (OS, libraries, configuration) into a **container**. A container is like a lightweight virtual computer that runs identically on any machine.

Key vocabulary:

- **Image** = The blueprint/recipe (like a class in OOP)
- **Container** = A running instance of an image (like an object)
- **Dockerfile** = Instructions to build an image (like a recipe)
- **docker-compose.yml** = Defines and runs multiple containers together

- **Volume** = Persistent storage that survives container restarts

Analogy: Think of a Docker image as a recipe for a cake. You can bake (run) that recipe to get a cake (container). You can bake the same recipe on any oven (machine) and get the same cake. If you want to keep the cake fresh between baking sessions, you put it in the fridge (volume).

6.2 Our Docker Setup

We have two files in the `docker/` folder:

6.2.1 `docker-compose.yml` — Defining Our Services

This file defines **two services** that work together:

```
version: "3.8"

services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: healthcare_sqlserver
    environment:
      ACCEPT_EULA: "Y"
      MSSQL_SA_PASSWORD: ${MSSQL_SA_PASSWORD:-YourStrong!Passw0rd}
      MSSQL_PID: Developer
    ports:
      - "1433:1433"
    volumes:
      - sqlserver_data:/var/opt/mssql
    healthcheck:
      test: /opt/mssql-tools18/bin/sqlcmd -S localhost
        -U sa -P "${MSSQL_SA_PASSWORD}"
        -C -Q "SELECT 1" -b -o /dev/null
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s

  data-generator:
    build:
```

```
context: ..
dockerfile: docker/Dockerfile.generator
container_name: healthcare_generator
depends_on:
  sqlserver:
    condition: service_healthy
env_file:
  - ../.env
ports:
  - "5000:5000"

volumes:
  sqlserver_data:
```

Listing 6.1: docker/docker-compose.yml

Line-by-line explanation:

1. **version:** "3.8" — The Docker Compose file format version

2. Service: sqlserver

- **image:** mcr.microsoft.com/mssql/server:2022-latest — Downloads the official SQL Server 2022 image from Microsoft
- **ACCEPT_EULA:** "Y" — Accepts the license agreement (required)
- **MSSQL_SA_PASSWORD** — The sa (system admin) password. Uses value from .env file, defaults to YourStrong!Passw0rd
- **MSSQL_PID:** Developer — Uses the free Developer Edition
- **ports:** "1433:1433" — Maps port 1433 in the container to port 1433 on your machine (1433 is the standard SQL Server port)
- **volumes:** sqlserver_data — Stores database files in a persistent volume so data survives container restarts
- **healthcheck** — Runs a SQL query (SELECT 1) every 10 seconds to verify the database is ready. Other services wait for this before starting.

3. Service: data-generator

- **build:** context/dockerfile — Builds from Dockerfile.generator (explained next)
- **depends_on:** service_healthy — Waits until SQL Server's healthcheck passes before starting

- `env_file: ../.env` — Loads environment variables from the `.env` file
- `ports: "5000:5000"` — Exposes the Flask API on port 5000

6.2.2 Dockerfile.generator — Building the Python Image

```
FROM python:3.11-slim

RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        curl \
        gnupg2 \
        unixodbc-dev && \
    curl -fsSL https://packages.microsoft.com/keys/microsoft.asc \
    | gpg --dearmor -o /usr/share/keyrings/microsoft-prod.gpg && \
    curl -fsSL https://packages.microsoft.com/config/debian/12/prod.list \
    | tee /etc/apt/sources.list.d/mssql-release.list && \
    apt-get update && \
    ACCEPT_EULA=Y apt-get install -y msodbcsql18 && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src/ ./src/
COPY config/ ./config/

EXPOSE 5000

CMD ["python", "src/generators/provider_api.py"]
```

Listing 6.2: docker/Dockerfile.generator

What each instruction does:

1. `FROM python:3.11-slim` — Start from a minimal Python 3.11 image (“slim” means fewer unnecessary packages, smaller image)

2. RUN `apt-get ...` — Install system dependencies:
 - `curl`, `gnupg2` — tools to download and verify Microsoft’s package signing key
 - `unixodbc-dev` — ODBC driver manager (needed to connect to SQL Server)
 - `msodbcsql18` — Microsoft’s ODBC Driver 18 for SQL Server
3. WORKDIR `/app` — Set the working directory inside the container
4. COPY `requirements.txt` & `pip install` — Install Python dependencies *before* copying source code (this is a Docker best practice — if your code changes but dependencies do not, Docker reuses the cached layer)
5. COPY `src/` & `config/` — Copy your application code into the container
6. EXPOSE 5000 — Document that this container listens on port 5000
7. CMD — The default command when the container starts

 Tip

Docker Layer Caching: Each instruction in a Dockerfile creates a “layer.” Docker caches layers and only rebuilds from the first changed instruction downward. By copying `requirements.txt` and installing dependencies *before* copying source code, we avoid reinstalling all packages every time we change a line of Python code. This makes builds much faster during development.

6.3 How to Use Docker in This Project

```
# Start both services (SQL Server + data generator)
docker-compose -f docker/docker-compose.yml up -d

# Check if services are running
docker ps

# View logs from the data generator
docker logs healthcare_generator

# View logs from SQL Server
docker logs healthcare_sqlserver

# Stop all services
docker-compose -f docker/docker-compose.yml down
```

```
# Stop and DELETE all data (fresh start)
docker-compose -f docker/docker-compose.yml down -v
```

Listing 6.3: Docker commands for local development

⚠ Warning

The `-v` flag in the last command deletes the persistent volume, which means **all database data is lost**. Only use it when you want a completely fresh start.

✏ Try It Yourself

Practice:

1. Run `docker ps` on your machine. What containers are currently running (if any)?
2. In the `docker-compose.yml`, what would happen if you removed the `healthcheck` section? (*Hint: think about the **depends_on** condition*)
3. The Dockerfile installs the ODBC driver for SQL Server. Why is this needed inside the Python container?

✓ Chapter Summary

Key Takeaways:

- Docker packages applications with all dependencies into portable containers
- Images are blueprints; containers are running instances; volumes persist data
- Our setup runs SQL Server 2022 and a Python data generator as two linked services
- The healthcheck ensures SQL Server is ready before the generator starts
- Docker layer caching speeds up builds by reusing unchanged layers

Chapter 7

Configuration — Environment Variables and Dependencies

📖 Learning Objectives

By the end of this chapter, you will be able to:

- Explain why configuration should be separated from code
- Set up the `.env` file from the provided template
- Understand each environment variable group and what it connects to
- Install Python dependencies from `requirements.txt`
- Configure code quality tools (flake8, pytest) via `setup.cfg`

7.1 Why Environment Variables?

Hardcoding passwords and connection strings directly in your code is dangerous:

- If you push code to GitHub, everyone can see your passwords
- Different environments (development, staging, production) need different values
- Changing a password means changing code, rebuilding, and redeploying

Environment variables solve this by storing configuration *outside* your code, in a `.env` file that is **never committed to Git**.

The Twelve-Factor App Principle

This approach follows a well-known software engineering principle from the **Twelve-Factor App** methodology: “Store config in the environment.” The idea is that your code should be identical in development and production — only the *configuration* (connection strings, passwords, feature flags) should differ.

7.2 The .env.example File

This is the *template* that shows what variables you need to set. It is safe to commit because it does not contain real secrets.

```
# SQL Server (Local Development)
MSSQL_HOST=localhost
MSSQL_PORT=1433
MSSQL_DB=healthcare_oltp
MSSQL_USER=sa
MSSQL_SA_PASSWORD=YourStrong!Passw0rd
MSSQL_DRIVER=ODBC Driver 18 for SQL Server

# Azure Data Lake Storage Gen2
ADLS_ACCOUNT_NAME=your_storage_account
ADLS_ACCOUNT_KEY=your_account_key
ADLS_CONTAINER_BRONZE=bronze
ADLS_CONTAINER_SILVER=silver
ADLS_CONTAINER_GOLD=gold

# Azure Event Hubs
EVENT_HUB_CONNECTION_STRING=your_connection_string
EVENT_HUB_NAME=vitals-stream

# Azure Synapse Analytics
SYNAPSE_SERVER=your_workspace.sql.azuresynapse.net
SYNAPSE_DATABASE=healthcare_dw
SYNAPSE_USER=your_username
SYNAPSE_PASSWORD=your_password

# Databricks
DATABRICKS_HOST=https://your-workspace.azuredatabricks.net
DATABRICKS_TOKEN=your_token

# Flask API
FLASK_HOST=0.0.0.0
FLASK_PORT=5000
FLASK_DEBUG=False
```

Listing 7.1: .env.example — the template (safe to share)

Variable groups explained:

1. **SQL Server** — Connection to the local Docker SQL Server (OLTP source)
2. **Azure Data Lake** — Cloud storage credentials. Bronze/Silver/Gold are container names matching our Medallion layers
3. **Event Hubs** — Connection for the real-time vitals streaming pipeline
4. **Synapse** — Connection to the cloud data warehouse (Gold layer target)
5. **Databricks** — Connection to the Spark processing cluster
6. **Flask** — Configuration for the Provider API service

! Important

To set up your environment:

1. Copy the template: `cp .env.example .env`
2. Edit `.env` with your actual values
3. **Never** commit `.env` to Git (it is in `.gitignore`)

7.3 Python Dependencies (requirements.txt)

This file lists every Python package the project needs, with minimum version numbers:

```
# Data Processing
pandas>=2.1.0           # DataFrames for tabular data
numpy>=1.25.0           # Numerical operations
pyarrow>=14.0.0         # Read/write Parquet files

# Data Generation
faker>=20.0.0           # Generate fake data (names, dates, etc
                        .)

# Database (SQL Server)
sqlalchemy>=2.0.0       # Python SQL toolkit and ORM
pyodbc>=5.0.0           # ODBC database connector
pymssql>=2.2.11         # Alternative SQL Server connector

# API
flask>=3.0.0            # Lightweight web framework
requests>=2.31.0        # HTTP client library
```

```
# Spark (local development)
pyspark>=3.5.0           # Apache Spark Python API
delta-spark>=3.0.0       # Delta Lake for Spark

# Visualization
matplotlib>=3.8.0       # Static charts and plots
seaborn>=0.13.0          # Statistical visualization
plotly>=5.18.0           # Interactive charts

# EDA & Profiling
ydata-profiling>=4.6.0   # Auto-generate data reports

# Streaming
azure-eventhub>=5.11.0   # Azure Event Hubs client

# Azure SDK
azure-storage-file-datalake>=12.14.0 # Data Lake client
azure-identity>=1.15.0    # Azure authentication

# Testing
pytest>=7.4.0            # Test framework
flake8>=6.1.0            # Code style checker

# Streamlit (optional)
streamlit>=1.29.0        # Web dashboard framework
```

Listing 7.2: requirements.txt — organized by purpose

To install all dependencies:

```
pip install -r requirements.txt
```

Tip

The `>=` syntax means “this version or newer.” For example, `pandas>=2.1.0` installs pandas 2.1.0 or any later version. This ensures you get bug fixes while maintaining compatibility. For production projects, you might use `==` (exact version) for fully reproducible builds.

7.4 Code Quality Configuration (setup.cfg)

```
[flake8]
max-line-length = 120
exclude = .git,__pycache__,venv,.venv,data
ignore = E203,W503

[tool:pytest]
testpaths = tests
python_files = test_*.py
python_functions = test_*
```

Listing 7.3: setup.cfg — linting and testing rules

- **flake8** — A Python code style checker (“linter”):
 - `max-line-length = 120` — Lines must not exceed 120 characters
 - `exclude` — Skip checking these folders
 - `ignore = E203, W503` — Ignore two specific style rules that conflict with the Black formatter
- **pytest** — The testing framework:
 - `testpaths = tests` — Look for tests in the `tests/` folder
 - `python_files = test_*.py` — Test files must start with `test_`
 - `python_functions = test_*` — Test functions must start with `test_`

✓ Chapter Summary

Key Takeaways:

- Never hardcode secrets — use environment variables stored in a `.env` file
- The `.env.example` is a safe template; the actual `.env` is Git-ignored
- `requirements.txt` lists all Python packages with minimum versions
- `setup.cfg` configures code linting (flake8) and test discovery (pytest)

Chapter 8

CI/CD Pipeline — Automated Quality Checks

Learning Objectives

By the end of this chapter, you will be able to:

- Define CI (Continuous Integration) and CD (Continuous Deployment)
- Read and understand a GitHub Actions workflow YAML file
- Explain each job in our CI pipeline and how they depend on each other
- Interpret CI pipeline results on GitHub (passing/failing checks)

8.1 What Is CI/CD?

CI/CD in Simple Terms

CI (Continuous Integration) means: every time someone pushes code, automated checks run immediately to catch problems early.

CD (Continuous Deployment/Delivery) means: if all checks pass, the code is automatically deployed.

Think of it as a quality control inspector on a factory assembly line — every product (code change) is checked before it moves to the next station.

We use **GitHub Actions** — a free CI/CD service built into GitHub.

Without CI: A developer pushes broken code → nobody notices → other developers pull broken code → everyone wastes time debugging.

With CI: A developer pushes broken code → automated tests fail within minutes → the developer is notified immediately → they fix it before anyone else is affected.

8.2 Our Pipeline Configuration

The pipeline is defined in `.github/workflows/ci.yml`:

```
name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  lint:
    name: Python Linting
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install flake8
      - run: flake8 src/ tests/ --max-line-length=120
        --exclude=__pycache__

  test:
    name: Unit Tests
    runs-on: ubuntu-latest
    needs: lint
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: pytest tests/ -v --tb=short

  sql-lint:
    name: SQL Validation
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install sqlfluff
      - run: sqlfluff lint sql/ --dialect tsql || true
```

Listing 8.1: .github/workflows/ci.yml — our CI pipeline

8.3 Pipeline Flow Explained

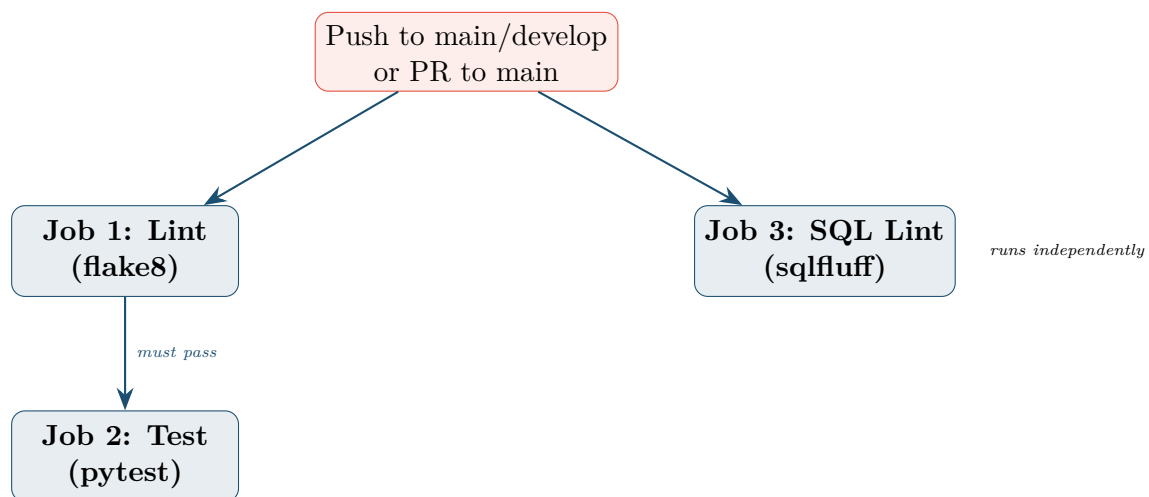


Figure 8.1: CI Pipeline execution flow

8.3.1 Job 1: Python Linting (flake8)

- **Triggers:** Runs on every push and PR
- **What it does:** Checks Python code style — line length, unused imports, syntax errors
- **If it fails:** Job 2 (tests) will not run

8.3.2 Job 2: Unit Tests (pytest)

- **Triggers:** Only runs *after* linting passes (`needs: lint`)
- **What it does:** Runs all tests in `tests/` folder
- **Flags:** `-v` (verbose output), `-tb=short` (short error tracebacks)

8.3.3 Job 3: SQL Validation (sqlfluff)

- **Triggers:** Runs in parallel with Job 1 (independent)
- **What it does:** Checks SQL files for style and syntax issues using T-SQL dialect

- **Note:** The `|| true` means it will not fail the pipeline even if SQL issues are found (soft enforcement for now)

Note

Reading the YAML file:

- **on:** defines *when* the pipeline runs (push to main/develop, or PR to main)
- **jobs:** defines *what* runs. Each job runs on a fresh virtual machine (ubuntu-latest)
- **steps:** are the sequential commands within a job
- **needs:** `lint` means “wait for the lint job to finish successfully first”
- **uses:** refers to pre-built GitHub Actions (like “checkout code” or “install Python”)

Chapter Summary

Key Takeaways:

- CI automatically runs checks (linting, tests) on every code push to catch bugs early
- Our pipeline has 3 jobs: Python linting, unit tests, and SQL validation
- Jobs can depend on each other (**needs:**) or run in parallel
- GitHub Actions workflows are defined in YAML files under `.github/workflows/`

Chapter 9

Testing — Verifying the Project Works

Learning Objectives

By the end of this chapter, you will be able to:

- Explain why automated testing is important in software projects
- Read and understand pytest test functions
- Run tests locally and interpret the output
- Understand what each test in our suite verifies

9.1 Why Test?

Testing prevents the “it works on my machine” problem. Automated tests:

- Catch bugs *before* they reach production
- Give you confidence to change code without breaking things
- Serve as documentation — tests show how code is *supposed* to work
- Run automatically in CI (Chapter 8) on every push

Types of Tests

- **Unit tests** — Test a single function or method in isolation. Fast, focused. *Example: “Does the date parser handle invalid dates correctly?”*
- **Integration tests** — Test that multiple components work together. *Example: “Can the generator write data to SQL Server successfully?”*
- **End-to-end tests** — Test the entire system from input to output. *Example: “Does data flow from Bronze to Gold correctly?”*

Our current test suite focuses on **scaffold validation** — verifying the project structure is correct. As the project grows, we will add unit and integration tests for each component.

9.2 Our Test Suite

Currently, the project has **scaffold validation tests** that verify the project structure is set up correctly. These run in CI on every push.

```
import os

ROOT_DIR = os.path.dirname(
    os.path.dirname(os.path.abspath(__file__))
)

def test_required_directories_exist():
    """Verify all expected project directories exist."""
    expected_dirs = [
        "src",
        "src/generators",
        "src/synthea",
        "notebooks",
        "sql",
        "sql/ddl",
        "sql/queries",
        "pipelines",
        "dashboards",
        "docs",
        "tests",
        "docker",
        "config",
    ]
    for d in expected_dirs:
        path = os.path.join(ROOT_DIR, d)
        assert os.path.isdir(path), f"Missing directory: {d}"

def test_required_files_exist():
    """Verify all critical files are present."""
    expected_files = [
        "README.md",
        "requirements.txt",
        ".env.example",
        ".gitignore",
        "setup.cfg",
    ]
```

```
        "docker/docker-compose.yml",
        "docker/Dockerfile.generator",
        ".github/workflows/ci.yml",
    ]
    for f in expected_files:
        path = os.path.join(ROOT_DIR, f)
        assert os.path.isfile(path), f"Missing file: {f}"

def test_env_example_has_required_keys():
    """Verify .env.example has all required variables."""
    env_path = os.path.join(ROOT_DIR, ".env.example")
    with open(env_path, "r") as f:
        content = f.read()

    required_keys = [
        "MSSQL_HOST",
        "MSSQL_PORT",
        "MSSQL_DB",
        "ADLS_ACCOUNT_NAME",
        "EVENT_HUB_CONNECTION_STRING",
        "SYNAPSE_SERVER",
        "FLASK_PORT",
    ]

    for key in required_keys:
        assert key in content, f"Missing env key: {key}"

def test_requirements_has_core_packages():
    """Verify requirements.txt has essential packages."""
    req_path = os.path.join(ROOT_DIR, "requirements.txt")
    with open(req_path, "r") as f:
        content = f.read()

    core_packages = [
        "pandas",
        "pyspark",
        "faker",
        "sqlalchemy",
        "pyodbc",
        "flask",
    ]
```

```
    "pytest",
]
for pkg in core_packages:
    assert pkg in content, f"Missing package: {pkg}"
```

Listing 9.1: tests/test_project_structure.py

9.2.1 Understanding Each Test

Test 1: test_required_directories_exist() — Loops through 13 expected directories and checks each one exists using `os.path.isdir()`. If any is missing, the test fails with a clear message like “Missing directory: sql/ddl”.

Line-by-line breakdown:

- `ROOT_DIR` is calculated by going up two levels from the test file to reach the project root
- `os.path.join(ROOT_DIR, d)` builds the full path to each directory
- `assert os.path.isdir(path)` checks the directory exists — if not, the test fails with the message after the comma

Test 2: test_required_files_exist() — Same pattern but for 8 critical files (README, requirements, Docker files, CI config). Uses `os.path.isfile()` instead of `isdir()`.

Test 3: test_env_example_has_required_keys() — Reads the entire `.env.example` file as a string, then checks that 7 key variable names appear in the text. This ensures the template is complete.

Test 4: test_requirements_has_core_packages() — Reads `requirements.txt` and checks that 7 essential package names are present. This ensures no one accidentally deletes a dependency.

9.3 Running Tests Locally

```
# Run all tests with verbose output
pytest tests/ -v

# Run a specific test file
pytest tests/test_project_structure.py -v

# Run a specific test function
```

```
pytest tests/test_project_structure.py::
    test_required_files_exist -v

# Run with short error output
pytest tests/ -v --tb=short
```

Listing 9.2: How to run tests

Expected output when all tests pass:

```
tests/test_project_structure.py::
    test_required_directories_exist PASSED
tests/test_project_structure.py::test_required_files_exist
    PASSED
tests/test_project_structure.py::
    test_env_example_has_required_keys PASSED
tests/test_project_structure.py::
    test_requirements_has_core_packages PASSED

===== 4 passed in 0.05s =====
```

💡 Tip

If any test fails, read the error message carefully. It will tell you exactly what is missing (e.g., “Missing directory: sql/ddl”). Create the missing item and re-run. The `-tb=short` flag gives you a concise error traceback without pages of detail.

✅ Chapter Summary

Key Takeaways:

- Tests catch bugs early, provide confidence for refactoring, and serve as documentation
- Our 4 scaffold tests verify: directories exist, files exist, `.env.example` is complete, and `requirements.txt` has all packages
- Use `pytest tests/ -v` to run all tests with verbose output
- Tests run automatically in CI on every push via GitHub Actions

Chapter 10

Git and Version Control

Learning Objectives

By the end of this chapter, you will be able to:

- Explain core Git concepts: repository, commit, branch, merge
- Understand what `.gitignore` and `.gitattributes` do
- Follow the feature-branch workflow used in this project
- Write proper commit messages using conventional prefixes

10.1 Git Basics for This Project

If you are new to Git, here is the minimum you need to know:

Git in 60 Seconds

Git tracks every change to every file in your project. Think of it as an “infinite undo” button with named checkpoints (called *commits*).

Key concepts:

- **Repository (repo)** = Your project folder, tracked by Git
- **Commit** = A snapshot of all files at a point in time (like a save point in a video game)
- **Branch** = A parallel version of the code (like a copy you can experiment with)
- **main** = The primary branch with stable, reviewed code
- **develop** = The integration branch where features are merged before going to main
- **Pull Request (PR)** = A request to merge your branch into main/develop, with code review

Visual analogy: Imagine a tree. The `main` branch is the trunk. When you create a feature branch, you grow a new limb. When the feature is done and reviewed, you merge it back into the trunk.

10.2 The .gitignore File

This file tells Git which files to **never track**. Our `.gitignore` excludes:

```
# Python caches (auto-generated, not needed)
__pycache__/
*.py[cod]

# Environment variables (contains secrets!)
.env
*.env.local

# Data files (too large for Git)
data/raw/*.csv
data/raw/*.json
data/raw/*.parquet
data/cleaned/*.parquet
data/gold/*.parquet

# Power BI files (binary, large)
*.pbix
*.pbit

# IDE settings (personal preference)
.vscode/
.idea/

# Virtual environment (can be recreated)
venv/
.venv/
```

Listing 10.1: Key sections of `.gitignore`

! Important

The most critical line is `.env` — this prevents your passwords and API keys from being pushed to GitHub where anyone can see them. **Always verify `.env` is in your `.gitignore` before pushing.**

10.3 The `.gitattributes` File

This file ensures consistent behavior across Windows, Mac, and Linux:

- **Text files** (Python, SQL, YAML, Markdown) — use LF (Unix) line endings
- **Windows scripts** (`.bat`, `.ps1`) — use CRLF (Windows) line endings
- **Binary files** (`.parquet`, `.pbix`, `.png`) — treated as binary (no line ending conversion)

💡 Tip

Windows uses CRLF (Carriage Return + Line Feed, two characters) for line endings, while Mac/Linux use just LF (one character). Without `.gitattributes`, Git might show every line as changed when switching between operating systems. This file prevents that confusion.

10.4 Common Git Workflow

```
# 1. Create a feature branch
git checkout -b feature/add-claims-generator

# 2. Make your changes...
# 3. Check what you changed
git status
git diff

# 4. Stage specific files
git add src/generators/generate_claims.py
git add tests/test_claims_generator.py

# 5. Commit with a descriptive message
git commit -m "feat: add claims data generator with Faker"

# 6. Push to remote
```

```
git push -u origin feature/add-claims-generator

# 7. Create a Pull Request on GitHub
# 8. After review and approval, merge to develop
```

Listing 10.2: Daily Git workflow

Note

Commit message prefixes:

- **feat:** — New feature
- **fix:** — Bug fix
- **docs:** — Documentation changes
- **test:** — Adding or updating tests
- **refactor:** — Code restructuring (no new features)
- **chore:** — Build process, dependencies, config changes

These prefixes follow the **Conventional Commits** standard and make it easy to scan the Git history and understand what each change did.

Chapter Summary

Key Takeaways:

- Git tracks all changes with commits (snapshots) and branches (parallel versions)
- **.gitignore** prevents secrets and large files from being committed
- **.gitattributes** ensures consistent line endings across operating systems
- Follow the feature-branch workflow: create branch → code → commit → push → PR → review → merge
- Use conventional commit prefixes (feat, fix, docs, test, refactor, chore)

Chapter 11

Analytical SQL Queries

Learning Objectives

By the end of this chapter, you will be able to:

- List the 10 analytical queries included in this project
- Explain window functions (LAG, RANK, NTILE) and how they differ from GROUP BY
- Read and understand a CTE-based query step by step
- Trace through the 30-day readmission rate query and explain each part
- Understand GROUPING SETS for multi-level aggregation

The project includes **10+ analytical queries** designed to answer real business questions. Each query demonstrates different SQL techniques. Here are all of them:

#	Query	SQL Techniques	Business Question
1	30-Day Readmission Rate	LAG(), DATEDIFF, CTE, CASE	Which conditions cause the most readmissions?
2	Patient Cost Distribution	NTILE(), PERCENTILE_CONT, window aggregation	How are costs distributed across patient segments?
3	Provider Performance	RANK(), DENSE_RANK(), AVG OVER	Which providers have the best outcomes?
4	Condition Co-occurrence	Self-JOIN, CROSS APPLY, PIVOT	Which conditions frequently appear together?
5	Monthly Encounter Trends	DATE_TRUNC, LAG(), running total	Are encounter volumes increasing or decreasing?

#	Query	SQL Techniques	Business Question
6	Claim Denial Analysis	GROUPING SETS, HAVING, ratios	What is the denial rate by payer?
7	Length of Stay Analysis	DATEDIFF, CASE, MEDIAN, outliers	Average length of stay by condition?
8	Patient Cohort Retention	Date cohort assignment, PIVOT	How do patient cohorts retain over time?
9	Top Medications by Cost	SUM, RANK(), cumulative % (Pareto)	Which medications drive the most cost?
10	Vital Signs Anomaly Summary	WHERE anomaly_flag=1, time patterns	When do most anomalies occur?

11.1 Key SQL Concepts Explained

Before diving into the queries, let us understand the key SQL concepts they use.

Window Functions — The Most Powerful SQL Feature

Window functions perform calculations across a set of rows *related to* the current row, without collapsing them into a single output row.

How they differ from GROUP BY:

- **GROUP BY** collapses rows: 1000 rows → 10 groups → 10 output rows
- **Window functions** keep all rows: 1000 rows → 1000 output rows, each with an extra computed column

Syntax pattern:

```
function_name() OVER (
    PARTITION BY column -- divide rows into groups
    ORDER BY column    -- order within each group
)
```

Common window functions:

- LAG(col, n) — Get the value from n rows *before* the current row

- **LEAD(col, n)** — Get the value from *n* rows *after* the current row
- **RANK()** — Assign a rank (1, 2, 2, 4 — ties get same rank, next rank is skipped)
- **DENSE_RANK()** — Like RANK but no gaps (1, 2, 2, 3)
- **NTILE(n)** — Divide rows into *n* equal groups (quartiles, deciles, etc.)
- **SUM() OVER()** — Running total or group total without collapsing

Simple example:

```
-- For each patient, show their visit date AND their
   previous visit date
SELECT
    patient_id,
    visit_date,
    LAG(visit_date, 1) OVER (
        PARTITION BY patient_id    -- group by patient
        ORDER BY visit_date        -- sort by date within
        each patient
    ) AS previous_visit_date
FROM encounters;
```

Result: each row now has both the current visit date and the previous one, making it easy to calculate gaps between visits.

Common Table Expressions (CTEs)

A **CTE** (Common Table Expression) is a named temporary result set that exists only for the duration of a single query. It is defined with **WITH ... AS (...)**.

Why use CTEs?

- Break complex queries into readable, named steps
- Each CTE can reference previous CTEs
- Much easier to debug than deeply nested subqueries

Simple example:

```
-- Without CTE (hard to read):
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER(PARTITION BY dept ORDER BY
    salary DESC) rn
```

```
        FROM employees
    ) sub WHERE rn = 1;

-- With CTE (easy to read):
WITH ranked_employees AS (
    SELECT *,
           ROW_NUMBER() OVER(PARTITION BY dept ORDER BY salary
                               DESC) AS rn
    FROM employees
)
SELECT * FROM ranked_employees WHERE rn = 1;
```

Both queries do the same thing (find the highest-paid employee in each department), but the CTE version clearly labels what each step does.

11.2 Deep Dive: 30-Day Readmission Rate

This query identifies patients who returned to the hospital within 30 days of discharge — a key quality metric in healthcare:

```
WITH encounter_pairs AS (
    SELECT
        fe.patient_key,
        fe.date_key AS current_visit,
        LAG(fe.date_key) OVER (
            PARTITION BY fe.patient_key
            ORDER BY dd.full_date
        ) AS previous_visit,
        dd.full_date AS visit_date,
        fe.length_of_stay_days
    FROM fact_encounters fe
    JOIN dim_date dd ON fe.date_key = dd.date_key
),
readmissions AS (
    SELECT *,
           DATEDIFF(DAY, previous_visit_date, visit_date)
           AS days_since_last_visit,
           CASE
               WHEN DATEDIFF(DAY, previous_visit_date, visit_date)
               <= 30
```

```

        THEN 1
        ELSE 0
      END AS is_readmission
FROM encounter_pairs
WHERE previous_visit IS NOT NULL
)
SELECT
  COUNT(*) AS total_encounters,
  SUM(is_readmission) AS readmissions,
  CAST(SUM(is_readmission) AS FLOAT) / COUNT(*)
    AS readmission_rate
FROM readmissions;

```

Listing 11.1: Readmission rate query pattern

Step-by-step walkthrough:

Step 1: CTE 1 — encounter__pairs: For each patient's encounters (sorted by date), use `LAG()` to look back at the *previous* visit date. This pairs each visit with the one before it.

What LAG does here: If patient #42 visited on Jan 5, Feb 10, and Mar 20, the result is:

visit_date	previous_visit	gap
Jan 5	NULL	—
Feb 10	Jan 5	36 days
Mar 20	Feb 10	38 days

Step 2: CTE 2 — readmissions: Calculate the number of days between consecutive visits using `DATEDIFF`. Then use `CASE` to flag any gap ≤ 30 days as a readmission (`is_readmission = 1`). Rows with no previous visit (`NULL`) are filtered out.

Step 3: Final SELECT: Count total encounters, sum the readmission flags, and divide to get the rate. `CAST(... AS FLOAT)` ensures decimal division (otherwise SQL does integer division: $3/10 = 0$ instead of 0.3).

💡 Tip

Why does this matter clinically? A high 30-day readmission rate may indicate:

- Patients being discharged too early
- Inadequate follow-up care after discharge

- Specific conditions that need better treatment protocols

Hospitals are sometimes penalized financially for high readmission rates, making this metric critically important.

✓ Chapter Summary

Key Takeaways:

- The project includes 10+ SQL queries covering readmissions, costs, provider performance, anomalies, and more
- Window functions (LAG, RANK, NTILE) compute values across related rows without collapsing them
- CTEs break complex queries into named, readable steps
- The readmission query uses LAG to pair consecutive visits, DATEDIFF to measure gaps, and CASE to flag readmissions

Chapter 12

Azure Cloud Services — What Each Service Does

Learning Objectives

By the end of this chapter, you will be able to:

- Name the 5 Azure services used in this project and explain each one's role
- Describe how data flows between Azure services
- Explain what a Synapse dedicated SQL pool is and why it uses 60 distributions
- Understand what columnstore indexes are and why they speed up analytics

This chapter explains each Azure service we use and how it fits into the pipeline. Do not worry about Azure pricing or setup details — focus on understanding *what each service does* and *why we chose it*.

12.1 Azure Data Lake Storage Gen2

- **What it is:** A cloud storage service optimized for big data analytics
- **Think of it as:** A huge, organized file system in the cloud (like a network drive with unlimited space)
- **Our use:** Store raw (Bronze), cleaned (Silver), and final (Gold) data files
- **Key feature:** Hierarchical namespace — you can organize files into folders, just like on your computer
- **Containers:** bronze, silver, gold (one per Medallion layer)

12.2 Azure Data Factory (ADF)

- **What it is:** A cloud-based data integration (ETL) service

- **Think of it as:** A delivery truck scheduler — it moves data between systems on a schedule
- **Our use:** Copy data from Data Lake (Bronze) into Databricks for processing
- **Key concepts:**
 - **Pipeline** = A workflow with multiple steps
 - **Activity** = A single step (e.g., Copy Data, Run Notebook)
 - **Trigger** = When to run (schedule, event-based)
 - **Linked Service** = Connection to an external system (Data Lake, Databricks, etc.)

12.3 Azure Databricks

- **What it is:** A managed Apache Spark platform
- **Think of it as:** A powerful computer cluster that processes massive datasets using PySpark
- **Our use:**
 - **Silver processing:** Cleaning, deduplication, schema enforcement
 - **Gold transformation:** Building the Star Schema, calculating KPIs
 - **Streaming:** Spark Structured Streaming for real-time vitals
- **Key feature:** Notebooks — interactive code environments similar to Jupyter, but running on a Spark cluster with distributed processing power

What Is Apache Spark?

Apache Spark is a distributed data processing engine. Instead of processing data on one computer, Spark splits the work across many computers (a “cluster”) working in parallel.

Why do we need it?

- Processing 500K encounters on one computer: maybe 10 minutes
- Processing 500K encounters on a 10-node Spark cluster: maybe 1 minute
- For real-world hospitals with billions of records, Spark is essential

PySpark is the Python API for Spark — you write Python code, and Spark distributes it across the cluster automatically.

12.4 Azure Event Hubs

- **What it is:** A big data streaming platform and event ingestion service
- **Think of it as:** A high-speed mail slot that can receive millions of messages per second
- **Our use:** Receives real-time vital sign readings from the IoT simulator
- **Key concept:** Producers (our Python simulator) *send* events; consumers (Spark Streaming) *read* events

12.5 Azure Synapse Analytics

- **What it is:** An enterprise analytics service (cloud data warehouse)
- **Think of it as:** A super-powered SQL Server designed for analytics queries on huge datasets
- **Our use:** Hosts the Gold layer Star Schema — Power BI connects here for dashboards
- **Key features:**
 - **Dedicated SQL Pool** — Reserved compute nodes always available
 - **60 distributions** — Data is automatically spread across 60 nodes for parallel processing
 - **Columnstore indexes** — Compresses data and speeds up analytical queries by 10–100x

What Are Columnstore Indexes?

Traditional databases store data **row by row** (like reading a book line by line). Columnstore indexes store data **column by column**.

Why does this matter for analytics?

When you run `SELECT AVG(total_cost) FROM fact_encounters`, the database only needs the `total_cost` column. With row storage, it must read *every column* of every row to find the one it needs. With columnstore, it reads *only* the `total_cost` column — skipping everything else.

Benefits:

- **10–100x faster** for analytical queries (reads less data)
- **10x better compression** (similar values in a column compress well)

- **Perfect for SUM, AVG, COUNT, GROUP BY** queries on large tables

12.6 How Services Connect

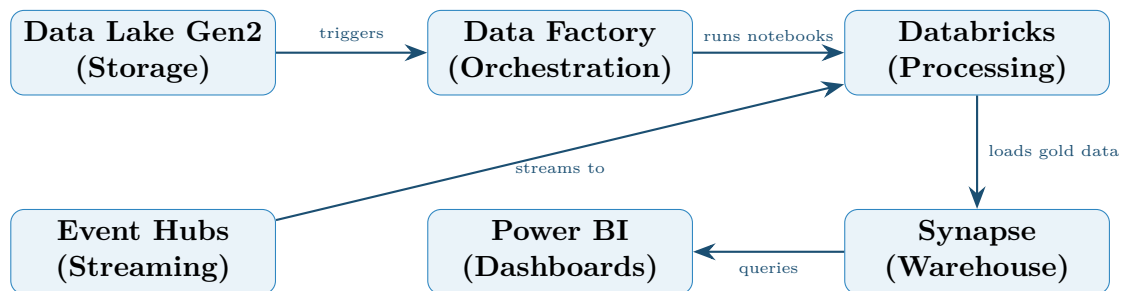


Figure 12.1: Azure service interconnections

✓ Chapter Summary

Key Takeaways:

- **Data Lake Gen2** = cloud file storage (Bronze/Silver/Gold containers)
- **Data Factory** = orchestration/scheduling (moves data between services)
- **Databricks** = Spark-based processing (cleaning, transformation, streaming)
- **Event Hubs** = real-time event ingestion (receives IoT vital signs)
- **Synapse** = cloud data warehouse (hosts Gold layer, serves Power BI)
- Columnstore indexes speed up analytical queries by reading only needed columns

Chapter 13

Getting Started — Step-by-Step Setup

Learning Objectives

By the end of this chapter, you will be able to:

- Install all prerequisites (Python, Git, Docker, Java, Azure CLI)
- Clone the repository and set up a virtual environment
- Configure environment variables and install dependencies
- Start Docker services and verify everything works
- Run the data generators

Follow these steps *in order* to set up the project on your machine.

13.1 Prerequisites

Before you begin, install these tools:

1. Python 3.11+ — <https://python.org/downloads>

- During installation on Windows, check “Add Python to PATH”

2. Git — <https://git-scm.com/downloads>

3. Docker Desktop — <https://docker.com/products/docker-desktop>

- On Windows, this requires WSL2 (Windows Subsystem for Linux)
- Docker Desktop will guide you through the WSL2 setup

4. Java 11+ (for Synthea) — <https://adoptium.net>

5. Azure CLI (for cloud deployment) — <https://aka.ms/installazurecliwindows>

6. A code editor — VS Code is recommended (<https://code.visualstudio.com>)

 Tip

You can verify each installation by running these commands in your terminal:

```
python --version      # Should show 3.11+
git --version          # Should show 2.x+
docker --version       # Should show 20.x+
java --version         # Should show 11+
az --version           # Should show Azure CLI version
```

13.2 Step 1: Clone the Repository

```
git clone https://github.com/ma7moudalysalem/healthcare-data-warehouse.git
cd healthcare-data-warehouse
```

 Note

If you do not have access to the repository, ask the team lead (Mahmoud Salem) to add your GitHub username as a collaborator.

13.3 Step 2: Create a Virtual Environment

A **virtual environment** isolates this project's Python packages from your system Python. This prevents conflicts between projects that need different package versions.

```
# Create the virtual environment
python -m venv venv

# Activate it
# On Windows (Command Prompt):
venv\Scripts\activate
# On Windows (PowerShell):
.\venv\Scripts\Activate.ps1
# On Mac/Linux:
source venv/bin/activate

# You should see (venv) at the start of your terminal prompt
```

13.4 Step 3: Install Python Dependencies

```
pip install -r requirements.txt
```

This installs all 20+ packages listed in `requirements.txt`. It may take a few minutes.

13.5 Step 4: Configure Environment Variables

```
# Copy the template
cp .env.example .env

# Edit with your favorite editor
code .env      # VS Code
notepad .env   # Windows Notepad
```

For local development, you only need to change the `MSSQL_SA_PASSWORD`. Azure variables are needed later for cloud deployment.

13.6 Step 5: Start Docker Services

```
# Start SQL Server and the data generator
docker-compose -f docker/docker-compose.yml up -d

# Verify both containers are running
docker ps

# Wait for SQL Server to be healthy (may take 30-60 seconds)
docker logs healthcare_sqlserver
```

13.7 Step 6: Run Tests

```
# Verify everything is set up correctly
pytest tests/ -v
```

You should see all 4 tests passing:

```
tests/test_project_structure.py::
    test_required_directories_exist PASSED
tests/test_project_structure.py::test_required_files_exist
    PASSED
tests/test_project_structure.py::
    test_env_example_has_required_keys PASSED
tests/test_project_structure.py::
    test_requirements_has_core_packages PASSED

===== 4 passed in 0.05s =====
```

Tip

If any test fails, read the error message carefully. It will tell you exactly what is missing (e.g., “Missing directory: sql/ddl”). Create the missing item and re-run.

13.8 Step 7: Run the Data Generators

```
# Generate synthetic claims data
python src/generators/generate_claims.py

# Start the vital signs simulator
python src/generators/simulate_vitals.py

# Start the provider API
python src/generators/provider_api.py
# API will be available at http://localhost:5000
```

13.9 Quick Reference Card

Command	What It Does
<code>pytest tests/ -v</code>	Run all tests
<code>flake8 src/ tests/</code>	Check code style
<code>docker-compose -f docker/</code> <code>docker-compose.yml up -d</code>	Start local services
<code>docker-compose -f docker/</code> <code>docker-compose.yml down</code>	Stop local services
<code>pip install -r requirements.txt</code>	Install dependencies
<code>python src/generators/</code> <code>provider_api.py</code>	Start the Flask API
<code>git status</code>	Check modified files
<code>git add . && git commit -m "msg"</code>	Save changes

✓ Chapter Summary

Key Takeaways:

- Setup follows 7 sequential steps: clone → venv → install → configure → Docker → test → generate
- Always use a virtual environment to isolate Python packages
- Docker services must be healthy before running data generators that depend on SQL Server
- All 4 tests should pass before proceeding to development

Chapter 14

Project Milestones and Timeline

Learning Objectives

By the end of this chapter, you will be able to:

- Describe the 5 project milestones and their deliverables
- Understand how each milestone builds on the previous one
- Identify which milestone you should be working on based on the timeline

The project is divided into **5 milestones** over **10 weeks**. Each milestone builds on the previous one.

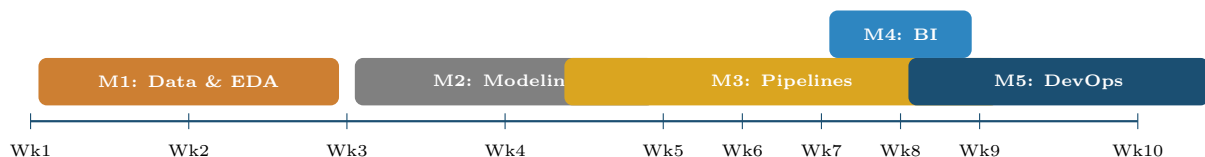


Figure 14.1: Project timeline — 5 milestones over 10 weeks

14.1 Milestone 1: Data Collection & EDA (Weeks 1–2)

Goal: Generate all data sources and understand the data through exploratory analysis.

Deliverables:

- ✓ Configure Synthea to generate 100K patient records
- ✓ Build the claims generator (`generate_claims.py`)
- ✓ Build the vitals simulator (`simulate_vitals.py`)
- ✓ Build the provider API (`provider_api.py`)
- ✓ Create an EDA notebook with 15+ visualizations
- ✓ Design the ER diagram for the OLTP schema
- ✓ Write SQL DDL scripts for source tables

14.2 Milestone 2: Data Modeling & Warehouse (Weeks 3–4)

Goal: Design and build the Star Schema in Azure Synapse.

Deliverables:

- ✓ Design the Star Schema (fact and dimension tables)
- ✓ Write Synapse DDL scripts with proper distribution strategies
- ✓ Implement SCD Type 2 for patient and provider dimensions
- ✓ Write 10+ analytical SQL queries
- ✓ Create the data dictionary (every column documented)

14.3 Milestone 3: Pipelines — Batch + Stream (Weeks 5–7)

Goal: Build the complete data pipeline from source to warehouse.

Deliverables:

- ✓ Configure Azure Data Factory pipelines (Copy Activities)
- ✓ Build Databricks PySpark notebooks for Silver layer processing
- ✓ Build Databricks notebooks for Gold layer transformation
- ✓ Implement Spark Structured Streaming for vital signs
- ✓ Build anomaly detection logic
- ✓ Verify Delta Lake tables with time travel queries

14.4 Milestone 4: BI Dashboards (Week 8)

Goal: Create interactive dashboards for business users.

Deliverables:

- ✓ Power BI Dashboard Page 1: Hospital Utilization
- ✓ Power BI Dashboard Page 2: Readmission Analysis
- ✓ Power BI Dashboard Page 3: Cost Analysis

- ✓ DAX measures for KPIs (readmission rate, average cost, etc.)
- ✓ Optional: Streamlit real-time monitoring dashboard

14.5 Milestone 5: DevOps & Presentation (Weeks 9–10)

Goal: Production-readiness, documentation, and final presentation.

Deliverables:

- ✓ CI/CD pipeline with GitHub Actions
- ✓ Docker containerization
- ✓ Azure Monitor and alerting setup
- ✓ Final project report
- ✓ Presentation slides

✓ Chapter Summary

Key Takeaways:

- The project follows 5 milestones: Data/EDA → Modeling → Pipelines → Dashboards → DevOps
- Each milestone has clear, measurable deliverables
- Milestones are sequential — you cannot build pipelines (M3) without the data model (M2)

Chapter 15

Team and Contribution Guidelines

Learning Objectives

By the end of this chapter, you will be able to:

- Identify all team members and their GitHub accounts
- Follow the contribution workflow from picking a task to merging a PR
- Write proper commit messages

15.1 Team Members

Name	Role	GitHub
Mahmoud Salem	Team Lead	@ma7moudalysalem
Ahmed Gamal Moussa	Member	@moussakii
Hussein Elsayed Gabr	Member	@Hussein-Gabr5757
John Emad Farhat	Member	@Johnemad791
Mahmoud Abdelaziz Tolan	Member	@Mahmoudtolan
Reem Ashraf Abdelbary	Member	@reemashr

15.2 How to Contribute

1. **Pick a task** from the project board or milestone list
2. **Create a branch** from develop: `git checkout -b feature/your-task`
3. **Write code** following the project conventions
4. **Write tests** for any new functionality
5. **Run checks** locally: `pytest tests/ -v` and `flake8 src/ tests/`
6. **Commit** with clear messages: `feat: add claims generator`
7. **Push and create a PR** targeting develop

8. **Request review** from at least one teammate
9. **After approval**, merge and delete the branch

✓ Chapter Summary

Key Takeaways:

- The team has 6 members working under the DEPI program
- Follow the contribution workflow: branch → code → test → commit → PR → review → merge
- Always run tests and linting before pushing

Chapter 16

Conclusion and Learning Roadmap

16.1 What You Have Learned

Congratulations on making it through this guide! Let us recap the major concepts you have covered:

Chapter	Key Concepts Learned
1. Introduction	OLTP vs OLAP, data warehousing fundamentals
2. Project Structure	Code organization, Python packages, Git-ignored files
3. Architecture	Medallion Architecture (Bronze/Silver/Gold), batch vs streaming
4. Data Modeling	Star Schema, fact/dimension tables, SCD Types 1 & 2, surrogate keys
5. Data Sources	Synthetic data generation, medical coding standards, REST APIs
6. Docker	Containerization, images, volumes, docker-compose
7. Configuration	Environment variables, dependency management, code quality tools
8. CI/CD	Continuous Integration, GitHub Actions, automated quality checks
9. Testing	pytest, test types, scaffold validation
10. Git	Version control, branching, .gitignore, conventional commits
11. SQL Queries	Window functions, CTEs, readmission analysis
12. Azure Services	Data Lake, Data Factory, Databricks, Event Hubs, Synapse
13–15	Setup, milestones, team workflow

16.2 Your Learning Roadmap

Now that you understand the project, here is a suggested learning path to deepen your skills. Each level builds on the previous one.

16.2.1 Level 1: Foundations (Start Here)

If you are new to data engineering, master these fundamentals first:

- 1. SQL Mastery** — Practice window functions, CTEs, JOINS, and GROUP BY until they are second nature
 - Resource: *SQLZoo* (free online exercises), *LeetCode SQL problems*
 - Goal: Be able to write the 10 analytical queries from Chapter 11 from scratch
- 2. Python for Data** — Learn pandas, file I/O, and basic data manipulation
 - Resource: *Python for Data Analysis* by Wes McKinney
 - Goal: Read a CSV, clean it, and write it as Parquet
- 3. Git Workflow** — Practice branching, merging, and resolving conflicts
 - Resource: *Learn Git Branching* (interactive online tutorial)
 - Goal: Complete a full feature-branch workflow independently

16.2.2 Level 2: Core Data Engineering

Once foundations are solid, learn the core tools:

- 1. Data Modeling** — Understand dimensional modeling (Kimball methodology)
 - Resource: *The Data Warehouse Toolkit* by Ralph Kimball
 - Goal: Design a star schema for a different domain (e.g., e-commerce, education)
- 2. PySpark** — Learn distributed data processing
 - Resource: Official PySpark documentation and Databricks Academy (free courses)
 - Goal: Write a PySpark notebook that reads, cleans, and transforms a CSV dataset
- 3. Docker** — Practice containerizing applications
 - Resource: Docker official “Get Started” tutorial
 - Goal: Dockerize a simple Python application from scratch

16.2.3 Level 3: Cloud and Production

Ready for cloud deployment and production systems:

1. **Azure Fundamentals** — Get the AZ-900 certification (free learning paths on Microsoft Learn)
2. **Azure Data Engineer** — Work toward the DP-203 certification
 - Covers: Data Lake, Data Factory, Databricks, Synapse — exactly what this project uses
3. **Streaming** — Study Spark Structured Streaming and event-driven architectures
4. **Data Quality** — Learn about Great Expectations, data contracts, and observability

16.2.4 Level 4: Advanced Topics

For those who want to go further:

1. **Data Mesh** — Decentralized data ownership at scale
2. **dbt (data build tool)** — SQL-based transformation framework
3. **Apache Kafka** — Production-grade streaming (Event Hubs is Kafka-compatible)
4. **MLOps** — Operationalizing machine learning models on data warehouse outputs
5. **Data Governance** — Cataloging, lineage, access control, and compliance (especially HIPAA for healthcare)

16.3 Final Words

This project demonstrates the **complete lifecycle** of a modern data engineering solution: from generating data, through cleaning and transforming it, to serving it in interactive dashboards. The skills you build here — SQL, Python, cloud services, data modeling, CI/CD — are the exact same skills used by data engineers at companies worldwide.

The best way to learn is to **build**. Pick a milestone, read the relevant chapters, and start coding. When you get stuck, re-read the concept boxes, ask your teammates, and consult the glossary.

! Important

Remember: Every expert was once a beginner. The concepts in this guide might feel overwhelming at first, but each one clicks into place as you practice. Focus on understanding *one layer at a time*, and the full picture will emerge.

Chapter 17

Glossary

A quick reference of terms used throughout this project. If you encounter an unfamiliar term, check here first. Terms are listed alphabetically.

Term	Definition
3NF	Third Normal Form — a database design that minimizes data redundancy by organizing into related tables
ACID	Atomicity, Consistency, Isolation, Durability — properties that guarantee reliable database transactions
ADF	Azure Data Factory — cloud ETL orchestration service
API	Application Programming Interface — a way for programs to communicate with each other
Batch Processing	Processing data in large chunks on a schedule (e.g., daily)
Bronze Layer	First layer of Medallion Architecture: raw, unmodified data
Business Key	The natural identifier from the source system (e.g., patient_id UUID)
CI/CD	Continuous Integration / Continuous Deployment — automated testing and deployment
Columnstore Index	A storage format that stores data by column instead of by row, optimized for analytics
CTE	Common Table Expression — named temporary result set in SQL
DAX	Data Analysis Expressions — formula language used in Power BI
DDL	Data Definition Language — SQL commands that define table structure
Delta Lake	Storage format with ACID transactions, schema enforcement, and time travel

Term	Definition
DML	Data Manipulation Language — SQL commands that work with data
ETL	Extract, Transform, Load — the process of moving data between systems
Fact Table	Central table in a Star Schema containing measurable events/metrics
Dimension Table	Descriptive context table in a Star Schema (who, what, when, where)
Gold Layer	Final layer of Medallion Architecture: business-ready, star schema data
Grain	The level of detail in a fact table (what one row represents)
HIPAA	Health Insurance Portability and Accountability Act — US healthcare data privacy law
ICD-10	International Classification of Diseases, 10th revision — standard for diagnosis codes
IoT	Internet of Things — physical devices that send data over the internet
Lambda Architecture	A data processing pattern that uses both batch and streaming paths
Medallion Architecture	Data organization pattern with Bronze, Silver, and Gold layers
OLAP	Online Analytical Processing — database optimized for complex queries
OLTP	Online Transaction Processing — database optimized for daily operations
Parquet	Columnar file format optimized for analytics (smaller files, faster queries)
REST API	Representational State Transfer API — standard web API pattern using HTTP
RxNorm	Standard naming system for medications
SCD	Slowly Changing Dimension — strategy for handling changes in dimension tables
Silver Layer	Middle layer of Medallion Architecture: cleaned, validated data
SNOMED CT	Systematized Nomenclature of Medicine Clinical Terms

Term	Definition
Star Schema	Database design with central fact tables surrounded by dimension tables
Stream Processing	Processing data continuously as it arrives in real-time
Surrogate Key	An artificial key (integer) assigned by the warehouse, separate from the business key
Synthea	Open-source tool for generating realistic synthetic patient data
UUID	Universally Unique Identifier — 128-bit value guaranteed to be unique
Virtual Environment	An isolated Python installation for a specific project
Window Function	SQL function that performs calculation across related rows without collapsing them

*This document was generated for the Healthcare Data Warehouse & Analytics Platform
 DEPI — Digital Egypt Pioneers Initiative
 Microsoft Data Engineer Track — Capstone Project*

Version 1.0
 February 25, 2026